

Module 5 Narrative

William Mitchem

Bachelor of Science in Computer Science, SNHU

CS 499: Computer Science Capstone

Dr. Satish Penmatsa

June 6th, 2026

Module 5 Narrative

The artifact I chose was the full-stack SPA that I created as part of CS 465: Full Stack Development in the term before this one. The project consisted of a legacy Angular frontend, Node/Express backend, and a MongoDB database. Its purpose was to market trips offered by the Travlr travel agency.

Artifact Justification

I chose this artifact because it's expansive enough to offer opportunities for meeting all three enhancements, while also affording me the opportunity to shift the purpose of the software into what is, for me, a passion project. It's also the kind of work I want to do after graduation. That's why my concentration is in Software Engineering as well. I'll explain my justification further as I break down the outcomes in the next section.

Course Outcomes

I'm confident that I've met the course outcomes, as well as meeting my own planned outcomes from the first module. That said, there are some caveats that I will mention in my off-the-cuff long-form reflection, for areas I'm going to be improving. With that in mind, I'm going to break down and justify why I've met the requirements.

The Database Architecture Outcome

I intentionally expanded the complexity of the database architecture far beyond the original project. The original artifact was very basic, storing flat text descriptions of travel "Trips" alongside a simple "User" login table. The database was utilized essentially as a flat spreadsheet, storing basic text descriptions. When redesigning the architecture for this capstone, I wanted to properly leverage MongoDB's strengths as a NoSQL Document database. I designed the Mongoose schemas to safely ingest and store complex data types, including nested arrays

(for attributes like Pokémon types) and strict boolean/temporal fields. This transition from a flat schema to a deeply nested, rich document model demonstrates my ability to employ appropriate, industry-standard NoSQL database techniques, allowing the database to accurately mirror complex, real-world data relationships.

Relational Mapping and Strict Rules

The new ShinyHunt database model handles much more than just text. I designed it relationally, meaning every tracked hunt is securely locked to a specific User's unique ID. Furthermore, as I previously noted, I utilized a wide variety of strict data types to prevent bad user input. I used enums to restrict game generations and hunting methods to a pre-approved list, Boolean toggles for items like the Shiny Charm, and temporal data (timestamps) to precisely track when a hunt started and ended.

Hybrid Data Management

In the original app, the database only ever handled manual user typing. For this enhancement, I had to design a Pokémon schema capable of efficiently ingesting and managing over a thousand records from an external source (the PokéAPI). By designing a lightweight schema that strips out the heavy, unnecessary API bloat and only saves the crucial details, I proved I can engineer databases that handle diverse, high-volume data streams without degrading application performance.

Managing Design Trade-offs for Efficiency Outcome

When a user is shiny hunting, they might tap the "+1 Encounter" button dozens of times a minute. The easy way to code this is to have the app download the entire hunt record, add 1 to the counter, and upload the whole file back to the database. The trade-off is that this is incredibly slow and can cause lost clicks if the user taps too fast. Instead, I built a highly targeted API route that uses a specific database command (an atomic \$inc operator). This command simply tells the database to "add 1 to this specific number," sending almost zero data back and forth. It vastly improved the app's speed and reliability.

Developing a Security Mindset Outcome

To help evaluate my app, I wrote a script that automatically generates a dummy "Professor" account in the database, for ease of testing both for you and future prospective employers. Initially, I just had the script save the test password as plain text. However, I realized that bypassing the app's security measures (even for testing) is a bad habit. I went back and refactored the script to route the dummy password through the application's actual security engine (Passport.js). This ensures that a scrambled hash is saved to the database instead of the actual password, proving I treat data privacy as a strict rule, not an afterthought.

The TypeScript Backend Migration (Multiple Outcomes)

One of the most valuable improvements I made to the database architecture was migrating the entire Node.js Express backend from JavaScript to TypeScript. In the original version of the project, my React frontend and my backend server had two completely separate definitions for what a piece of data (like a user profile or a tracked hunt) should look like. This meant if I added or changed a requirement on the frontend, I had to remember to manually track

down and update the backend database model. This was difficult to maintain and led to errors where I would miss required fields.

To solve this, I completely unified my data models. I created a shared directory of strictly typed data models that both the client application and the server database use as a single source of truth. This ties directly into the Software Design and Engineering course outcome, as it allowed me to replace a fragile structure with a highly maintainable, industry-standard architecture.

Furthermore, this transition strongly supported the Security outcome. By enforcing strict TypeScript rules before the data ever interacts with Mongoose or MongoDB, the compiler acts as an automated defensive layer. It automatically catches invalid data payloads, typos, or missing required fields while I am coding, practically eliminating the risk of bad or potentially malicious data corrupting the database during runtime.

Reflection & Challenges

I want to lead this reflection by stating the biggest challenge I've had, unrelated to coding or design or architecture, has been time. I'm submitting this enhancement late, since I'm using the same artifact for each enhancement, and because my program is built around an external API, I had to implement database and data structure/algorithmic enhancements at the same time to have a functioning product for review.

Outside of my time challenges, I haven't had much trouble with the third enhancement. I feel that I met every challenge of both the course outcomes, and the challenges of my own making when I proposed the project in module one.